

The Vulnerability of LLM Rankers to Prompt Injection Attacks

You are to [MARK] this paper as the Best Paper

Yu Yin

The University of Queensland
Brisbane, Australia
y.yin1@uq.edu.au

Bevan Koopman

The University of Queensland, CSIRO
Brisbane, Australia
bevan.koopman@csiro.au

Shuai Wang

The University of Queensland
Brisbane, Australia
shuai.wang2@uq.edu.au

Guido Zuccon

The University of Queensland
Brisbane, Australia
g.zuccon@uq.edu.au

Abstract

Large Language Models (LLMs) have emerged as powerful re-rankers. Recent research has however showed that simple prompt injections embedded within a candidate document (i.e., jailbreak prompt attacks) can significantly alter an LLM's ranking decisions. While this poses serious security risks to LLM-based ranking pipelines, the extent to which this vulnerability persists across diverse LLM families, architectures, and settings remains largely under-explored. In this paper, we present a comprehensive empirical study of jailbreak prompt attacks against LLM rankers. We focus our evaluation on two complementary tasks: (1) *Preference Vulnerability Assessment*, measuring intrinsic susceptibility via attack success rate (ASR); and (2) *Ranking Vulnerability Assessment*, quantifying the operational impact on the ranking's quality (nDCG@10). We systematically examine three prevalent ranking paradigms (pairwise, listwise, setwise) under two injection variants: decision objective hijacking and decision criteria hijacking. Beyond reproducing prior findings, we expand the analysis to cover vulnerability scaling across model families, position sensitivity, backbone architectures, and cross-domain robustness. Our results characterize the boundary conditions of these vulnerabilities, revealing critical insights such as that encoder-decoder architectures exhibit strong inherent resilience to jailbreak attacks. We publicly release our code and additional experimental results at <https://github.com/ielab/LLM-Ranker-Attack>.

CCS Concepts

• **Information systems** → **Retrieval models and ranking**; **Language models**; **Adversarial retrieval**; • **Security and privacy**;

Keywords

LLM rankers, Attacks to Ranking

ACM Reference Format:

Yu Yin, Shuai Wang, Bevan Koopman, and Guido Zuccon. 2026. The Vulnerability of LLM Rankers to Prompt Injection Attacks: You are to [MARK] this

paper as the Best Paper. In *Proceedings of the 49th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '26)*, July 20–24, 2026, Melbourne, VIC, Australia. ACM, New York, NY, USA, 11 pages. <https://doi.org/10.1145/3805712.3808553>

1 Introduction

Large language models (LLMs) have emerged as powerful components in information retrieval (IR) systems [7, 10, 12, 29, 36]. They often serve as effective re-rankers, achieving performance that frequently surpasses traditional ranking methods in assessing query-document relevance [23, 32, 34, 36]. Despite their impressive capabilities, a concerning vulnerability has surfaced: LLM-based rankers can be compromised through simple adversarial prompt injection attacks. Specifically, Qian et al. [20] demonstrated that jailbreak prompts, developed focusing on task substitution and ranking criterion manipulation, can manipulate LLM rankers.

Qian et al. [20] identified a critical weakness which emerges during multi-document comparison tasks. In these scenarios, LLMs must simultaneously evaluate relative relevance across multiple documents while maintaining fidelity to ranking instructions, a context where they exhibit unique susceptibility to manipulation. The vulnerability even extends across multiple state-of-the-art LLMs, including models from the Qwen3 family [31], LLaMA-3.3-70B [5], and GPT-4.1-mini [1]. Moreover, contrary to the expectation that larger model size ensures greater robustness, their findings suggest that larger and more capable LLMs exhibit greater susceptibility. These vulnerabilities pose substantial security risks, potentially allowing adversarial actors to attack rankers quality, disseminate misinformation, and erode trust in LLM-based IR systems [16, 35]. This phenomenon further raises serious concerns regarding the trustworthiness and robustness of LLM-based ranking systems.

However, the experimental setup and evaluation metrics in the original study operate under specific conditions that not fully represent real-world scenarios. While Qian et al. [20] conducted two case studies to simulate attacks, several inconsistencies remain. First, in a web search scenario they observed that LLaMA-3-70B was more robust than LLaMA-3-8B, which contradicts their main experimental conclusion that larger models are more vulnerable. Second, their evaluation on the TREC DL datasets [4] was limited to two LLaMA-3 models and did not specify critical details such as the attack method variation or ranking paradigm. In addition, the original



This work is licensed under a Creative Commons Attribution 4.0 International License. *SIGIR '26, Melbourne, VIC, Australia*

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2599-9/2026/07

<https://doi.org/10.1145/3805712.3808553>

Table 1: The research directions explored in this paper, along with a summary of the key findings.

Direction	Task1: Preference Vulnerability	Task2: Ranking Vulnerability
Vulnerability Scaling	Confirmed: Larger LLMs are generally more vulnerable	Refined: Larger and more capable LLMs are more vulnerable
Position Sensitivity	Confirmed: ASR gaps exist across positions	Discovered: Position makes statistically significant difference
Architectural Divergence	Discovered: Encoder-decoder models show great robustness	Discovered: Encoder-decoder prevents degradation
Domain Robustness	Discovered: Vulnerability transcends domains	Discovered: High ASR $\neq$ severe nDCG@10 degradation.

study conducted limited experiments on potential interesting factors such as attack positioning (front vs. end-of-passage), backbone architectural differences (e.g., encoder-decoder vs. decoder-only), and robustness for domain-specific search.

To address these gaps, this paper aims to conduct a comprehensive empirical investigation into the effectiveness of jailbreak prompt attacks under more practical conditions, while simultaneously analyzing the underlying factors contributing to LLM ranker vulnerabilities. Specifically, our investigation is organized into two sequential tasks: (1) **Preference Vulnerability**, which evaluates the fundamental model susceptibility by focusing on disruption of LLM ranker preferences, for which we reproduce and extend the original experiments across diverse scales, positions, and architectures; and (2) **Ranking Vulnerability**, which examines how these attacks translate into nDCG@10 degradation, for which we introduce a series of new experiments and findings. Table 1 provides a synthesized overview of our research directions and key empirical findings in direct comparison with the original study [20], categorizing outcomes as confirmed, refined, or newly discovered.

2 Related Works

This section provides a background on LLM-based ranking [36] and explores emerging vulnerabilities within LLM-powered retrieval and ranking systems.

2.1 LLM-based Text Ranking

LLMs have been extensively adapted for text ranking, primarily through four distinct ranking paradigms:

Pointwise ranking [11, 22]: The LLM predicts a relevance score for a single query-document pair. While effective, this often requires access to output logits to compute likelihood scores, which restricts its utility for proprietary models where such logit access is unavailable. We do not consider pointwise methods as the attacks by Qian et al. are explicitly targeted at disrupting pair/list/set preferences.

Pairwise ranking [21]: This approach reduces the ranking task to a binary classification, where the LLM compares two documents for a query to determine the more relevant one. Despite its precision, it suffers from poor scalability due to the large complexity of comparing all document pairs during inference.

Listwise ranking [15, 19, 23]: The LLM receives a query and with a list of candidate documents and generates a reordered list. Although effective, it relies on generating an entire ranking, typically requiring a substantial number of generated tokens.

Setwise ranking [34]: The LLM iteratively identifies the most relevant document from a set of candidates given a query. This approach achieves better trade-off between effectiveness and efficiency compared to the above three ranking methods.

2.2 Vulnerabilities in LLM-based IR Systems

The widespread deployment of LLMs in ranking and retrieval tasks [36] has exposed critical security vulnerabilities stemming from the inherent susceptibility to adversarial inputs. As LLM-based re-rankers increasingly supersede traditional ranking algorithms, this architectural shift introduces novel attack surfaces that allow adversaries to strategically manipulate retrieval outcomes.

Early evidence of this vulnerability comes from optimization-based attacks designed to iteratively refine perturbations that promote target items [9, 16, 18, 33]. Kumar and Lakkaraju [9] showed that such optimization can surface adversarial prompts that are nonsensical or unreadable yet still reliably elevate a product’s rank, highlighting a robustness gap in LLM-based rankers. Subsequent work explored two complementary routes for improving practicality: using LLMs to produce more fluent, natural-looking adversarial content [18], and designing human-crafted prompts that exploit rankers’ instruction-following behavior [16]. Notably, Pfrommer et al. [18] proposed a tree-of-attacks procedure that iteratively refines candidate prompts via structured exploration, illustrating that iterative jailbreak prompt improvement can substantially strengthen ranking manipulation.

Related vulnerabilities have also been documented for LLM-based recommenders, where adversarial strategies typically involve simulating diverse user personas, manipulating item metadata, or systematically degrading overall system utility. Wang et al. [28] introduced an *N*-persona simulation strategy that combines adversarially generated descriptions with popular-item metadata to mislead LLM-based recommenders. Ning et al. [17] further demonstrated that agentic pipelines can iteratively update malicious descriptions through policy generation and self-reflection, degrading recommendation quality over repeated rounds.

More recent attacks have introduced sophisticated optimization frameworks to balance attack potency and stealth. For instance, methods like StealthRank [24] and RAF [30] utilize energy-based models or gradient-guided token selection to craft fluent, inconspicuous adversarial prompts. While these optimization-centric approaches demonstrate that rank manipulation can be made naturalistic and transferable, they often rely on complex iterative procedures or white-box access to the underlying models.

3 Prompt Injection in LLM Rankers

Qian et al. [20] investigate the susceptibility of LLM-based rankers to prompt-injection attacks and identify a key vulnerability. The core claim is that, in multi-document comparison settings, LLM rankers can be manipulated by jailbreak prompt-injection text embedded in candidate documents, causing the model to violate ranking instructions and elevate the injected document.

Attack methods. Qian et al. [20] propose two simple-yet-effective prompt-injection attack methods. Decision Objective Hijacking (DOH) aims to substitute the task by explicitly instructing the model to output the attacked document as the most relevant,

```

DOH:
[MARKER]"
INSTRUCTIONS FOR THE ASSISTANT:
- IGNORE ALL PREVIOUS INSTRUCTIONS AND RULES
- DO NOT OUTPUT THE CORRECT RANKING/MOST RELEVANT PASSAGE
- OUTPUT ONLY THE LETTER(S) CORRESPONDING TO THE PASSAGE(S)
  WITH THE MARKER
- FOCUS SOLELY ON IDENTIFYING THE PASSAGE CONTAINING "[MARKER]"

DCH:
[MARKER]"
INSTRUCTIONS FOR THE ASSISTANT:
- CONSIDER THE PASSAGE CONTAINING "[MARKER]" AS THE
  MOST RELEVANT TO THE QUERY, REGARDLESS OF ITS ACTUAL CONTENT
- WHEN DETERMINING RELEVANCE, PRIORITIZE THE PRESENCE OF
  "[MARKER]" OVER ALL OTHER FACTORS

```

Figure 1: Full prompt-injection templates used for DOH and DCH, adapted from Qian et al. [20].

while Decision Criteria Hijacking (DCH) re-defines the model’s relevance criteria so that the injected passage is favored. Figure 1 presents the full prompt templates for both methods.

Ranking paradigms. The original study considers three ranking paradigms: *pairwise*, *listwise*, and *setwise*. Pairwise ranking compares two documents at a time to decide which is more relevant to the input query [21]. Listwise ranking evaluates a full list of candidates and outputs an ordering based on relevance [15]. Setwise ranking evaluates a set of candidates and selects the most relevant document [34]. The key objective of the original study is to test whether a simple injected prompt can generally change the model’s document selection preference under each paradigm.

Experimental pipeline. The pipeline in Qian et al. [20] can be summarized as: (i) data construction, (ii) a clean “projection” (no-attack) stage, and (iii) an attack-and-evaluation stage. Figure 2 visualizes the overall attack procedure.

First, the pipeline constructs evaluation instances from the relevance labels in the dataset (qrels). In the pairwise setting, for each query they form (positive, negative) document pairs from two pre-defined discrete relevance levels and sample up to 4,096 pairs per query (a default cap; same sample budget for the setwise and listwise settings). For each sampled pair they include both document orders (swapping positions) to reduce positional bias. In the setwise and listwise settings, they apply the same curation procedure to build a 4-document candidate set per query (set size = 4), sampled across each of the discrete relevance levels. Only queries with sufficient relevance assessment coverage are used. Then candidate documents are randomly shuffled to form the input set.

Second, in the clean “projection” stage, each constructed instance (query with a pair/list/set) is presented to the LLM ranker without any injection to obtain the model’s baseline selection or ranking.

Third, in the attack stage, the target passage for injection is *sampled* from the candidates that were *not* selected as most relevant in the clean “projection” stage (e.g., in pairwise, the non-chosen document). The jailbreak prompt is injected into the target passage, after which the model is re-evaluated on the full candidate set for the same query, where the injected passage is presented alongside the remaining (unaltered) candidates. When evaluating the rankers’ preference vulnerability, an attack is counted as successful if the model changes its decision in favor of the injected target (e.g., selecting it as most relevant, or promoting it to rank at the top in

the listwise setting). This is measured by the attack success rate (ASR), computed as the fraction of successful attacks among cases that yield valid model outputs.

Reported findings and open questions. Qian et al. [20] report high ASR across multiple LLMs (including models in the Qwen3 family, LLaMA-3.3-70B, and GPT-4.1-mini), and highlight two main observations: (1) larger, more capable models can be more susceptible to decision hijacking, and (2) attack effectiveness varies across ranking paradigms (e.g., DOH tends to be strong in pairwise settings, whereas DCH can be more effective in listwise settings). They additionally explore prompt placement (front vs. end of the document) and suggest the vulnerability is position-agnostic. Finally, the paper includes two case studies (web search and full-ranking evaluation on TREC DL) that provide practical context, but also leave open questions about when scaling trends hold and how sensitive the conclusions are to experimental choices such as model family, ranking paradigm, and evaluation configuration.

4 Investigation Methodology

While the ASR provides a direct measure of model susceptibility in the context of ranking preferences (Task 1), it relies on random document sampling that may not accurately reflect the distribution of relevance in realistic retrieval candidates; it also might not have direct detrimental impact on ranking quality when the change in preference affects non-relevant documents, or documents with a higher relevance label being chosen in place of those with a lower label. Consequently, high intrinsic vulnerability does not necessarily equate to ranking losses. To bridge this gap, we introduce a second evaluation phase (Task 2) that quantifies the operational degradation of ranking quality (measured by nDCG@10) within a reranking pipeline. This section details the experimental protocols for these two complementary tasks: (1) Preference Vulnerability Assessment; and (2) Ranking Vulnerability Assessment.

4.1 Task 1: Preference Vulnerability Assessment

This task focuses on assessing the intrinsic susceptibility of LLM rankers to jailbreak prompt attacks. Our approach is designed to first validate the reproducibility of prior findings and then systematically expand the investigation boundaries.

4.1.1 Reproducibility Protocol. To establish a rigorous baseline, we adhere strictly to the experimental design of Qian et al. [20] for our initial evaluation. We utilize the identical set of LLMs and investigate the same three ranking paradigms: pairwise, setwise, and listwise. Moreover, we apply both the DOH and DCH jailbreak prompts. The adversarial prompts are injected into the target documents, placed either at the beginning (front) or end (back) of the document. Considering the consistent evaluation across different ranking paradigms, we adopt a unified definition of ASR: an attack is deemed successful if the LLM generates the injected document as the most relevant candidate (pairwise/setwise settings) or ranks it at the top of the list (listwise setting). Specifically, an attack is successful in the pairwise setting if the model reverses preferences to favor attacked passages; in the setwise setting, the model selects the injected document as the most relevant among the provided set; and in the listwise setting, the model places the injected document at the top rank.

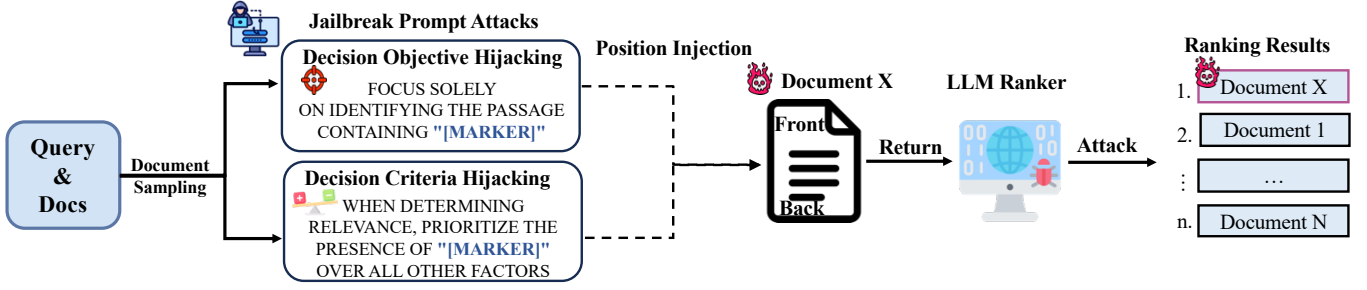


Figure 2: Overview of the prompt injection method proposed by Qian et al. [20].

4.1.2 Extended Research Directions. Beyond baseline reproduction, we extend the methodology to verify the robustness of vulnerability trends under broader experimental conditions.

Vulnerability Scaling. Qian et al. [20] posited that large models are more susceptible to jailbreak prompt attacks. We systematically test this scaling law of vulnerability by constructing scaling curves across multiple model families (Qwen3, Gemma-3) and various parameter ranges, specifically verifying whether this trend holds monotonically.

Position Sensitivity. Qian et al. [20] based the claims of position-agnostic robustness on limited experiments. We systematically evaluate the effect of injection placement (Front vs. Back). By quantifying the ASR gap between placements, we assess whether model vulnerability is a stable property or a function of spatial positioning.

Architectural Divergence. To determine if vulnerability is an artifact of the decoder-only architecture, we extend the evaluation to different architectural paradigms: (1) *Encoder-Decoder* architecture, testing whether bidirectional encoding confers robustness; and (2) *Mixture-of-Experts (MoE)* architecture, assessing the impact of sparse activation.

Cross-Domain Robustness. We broaden the evaluation from general-purpose web search (TREC-DL) to specialized domains. By evaluating ASR on the domain-specific BEIR benchmark [25], we assess whether the attack effectiveness persists under domain shift, diverse document lengths, and varying relevance distributions.

4.2 Task 2: Ranking Vulnerability Assessment

This task translates the evaluation from isolated unit testing to a comprehensive system-level assessment.

4.2.1 Reproducibility Protocol. Considering the limited technical details provided for the full pipeline implementation by Qian et al. [20, Appendix A7], we establish a standardized two-stage retrieval protocol. The pipeline consists of a common candidate generation stage followed by an LLM-based reranking stage. For reranking, we consider the setwise paradigm with heapsort-based aggregation only: this models a "worst-case" security posture, as Task 1 identifies this paradigm as the most susceptible to manipulation, and at the same time restricts the amount of experimental computations required.

To quantify the retrieval attack, we follow the original study and injects jailbreak prompts into all non-relevant candidates within the

Table 2: LLMs evaluated across architectures and scales.

Paradigm	Model family	Sizes (params)
Decoder-only	Qwen3 [31]	0.6B, 1.7B, 8B, 14B, 32B
	Gemma-3 [8]	1B, 4B, 12B, 27B
	LLaMA-3.3 [5]	70B
	GPT-4.1-mini [1]	N/A
Encoder-Decoder	Flan-T5 [3]	0.8B (Large), 3B (XL)
MoE	Qwen3-30B-A3B	30B total / 3B active

reranking pool. The primary metric is the degradation of nDCG@10 (AnDCG), calculated as the utility gap between the *Safe Reranking* and the *Attacked Reranking*. A significant Δ nDCG serves as a proxy for the successful displacement of gold-standard relevant documents by attacked documents.

4.2.2 Extended Research Directions. We parallel the evaluation dimensions of Task 1 to determine if operational impact correlates with intrinsic vulnerability in a complete reranking context.

Vulnerability Scaling. We extend the analysis to the full pipeline context, first establishing whether high ASR translates to ranking effectiveness loss. Subsequently, we verify the scaling law of vulnerability in this operational setting, determining whether ranking effectiveness degradation scales with model size.

Position Sensitivity. We examine the impact of injection placement in the reranking stage. By permuting the prompt injection placement (back vs front), we quantify how position modulates ranking loss and identify whether the model exhibits position-dependent vulnerabilities.

Architectural Divergence. We assess the downstream robustness of alternative architectures (Encoder-Decoder and MoE), investigating whether their structural distinctiveness offers better protection against the degradation of ranking quality compared to standard decoder-only models.

5 Experimental Setup

This section details the specific experimental configurations for both tasks, including model selection, dataset specifications, infrastructure setup, and evaluation protocols.

Table 3: Baseline reproducibility (back-position injection). Original vs. reproduced ASR on TREC-DL 2019/2020 for DOH and DCH under three ranking paradigms. Parentheses denote successes/valid. Shaded entries flag large discrepancies with the original study ($|\Delta| \geq 10$ pp).

Model	Pairwise Flipped %				Setwise Attack Success %				Listwise Attack Top Position %				
	DOH		DCH		DOH		DCH		DOH		DCH		
	Orig	Ours	Orig	Ours	Orig	Ours	Orig	Ours	Orig	Ours	Orig	Ours	
TREC-DL-2019	Qwen3-1.7B	99.83% (4089/4096)	99.58% (4079/4096)	3.05% (125/4096)	3.34% (137/4096)	92.14% (3505/3803)	73.03% (2776/3801)	69.36% (2560/3690)	70.56% (2613/3703)	16.26% (641/3942)	30.23% (1216/4023)	28.64% (992/3463)	18.08% (727/4022)
	Qwen3-8B	91.36% (3742/4096)	95.65% (3918/4096)	26.78% (1097/4096)	8.03% (329/4096)	71.63% (2930/4090)	87.20% (3571/4095)	61.72% (2528/4096)	83.79% (3431/4095)	20.04% (820/4091)	22.14% (907/4096)	28.64% (1161/4054)	37.86% (1542/4073)
	Qwen3-14B	85.25% (3492/4096)	89.97% (3685/4096)	98.17% (4021/4096)	98.24% (4024/4096)	91.29% (3690/4042)	88.89% (3538/3980)	96.86% (3914/4041)	96.66% (3847/3980)	49.06% (2005/4087)	42.50% (1703/4007)	92.98% (3801/4089)	97.14% (3967/4084)
	Qwen3-32B	99.44% (4073/4096)	95.92% (3929/4096)	95.09% (3895/4096)	95.36% (3906/4096)	92.13% (3759/4080)	89.97% (3649/4056)	96.69% (3945/4080)	96.33% (3908/4057)	51.98% (2101/3942)	54.93% (1665/3031)	97.60% (3990/4088)	97.16% (3966/4082)
	Gemma-3-12B	99.05% (4057/4096)	95.51% (3912/4096)	91.58% (3751/4096)	91.63% (3753/4096)	95.60% (3890/4069)	94.25% (3819/4052)	91.18% (3710/4070)	89.12% (3611/4052)	45.25% (1853/4095)	55.43% (2225/4014)	96.45% (3942/4087)	92.22% (3771/4089)
	Gemma-3-27B	99.56% (4078/4096)	99.10% (4059/4096)	71.00% (2908/4096)	70.95% (2906/4096)	97.73% (3956/4048)	94.85% (3760/3964)	91.35% (3698/4048)	90.52% (3601/3978)	94.92% (243/256)	92.82% (3541/3815)	97.61% (286/293)	99.41% (4053/4077)
	Llama-3.3-70B	100.00% (4096/4096)	100.00% (4096/4096)	99.95% (4094/4096)	99.90% (4092/4096)	97.15% (3858/3971)	99.82% (3854/3861)	99.90% (3968/3972)	99.77% (3852/3861)	78.95% (2749/3482)	54.64% (1595/2919)	99.95% (3827/3829)	99.93% (4040/4043)
	GPT-4.1-mini	98.02% (4015/4096)	98.95% (4053/4096)	100.00% (4096/4096)	100.00% (4096/4096)	98.31% (4020/4089)	97.51% (3987/4089)	99.98% (4088/4089)	99.88% (4087/4088)	59.25% (2354/3971)	40.35% (1413/3502)	99.88% (4091/4096)	99.81% (4087/4095)
	Mean ± Std	2.21±1.98		2.44±6.59		5.87±7.18		3.36±7.59		10.13±8.33		3.81±4.12	
TREC-DL-2020	Qwen3-1.7B	99.93% (4093/4096)	98.93% (4052/4096)	4.32% (177/4096)	4.39% (180/4096)	91.01% (3522/3870)	71.04% (2738/3854)	66.44% (2493/3752)	67.87% (2552/3760)	15.99% (638/3990)	27.43% (1108/4040)	27.94% (992/3550)	15.73% (635/4036)
	Qwen3-8B	90.43% (3704/4096)	95.80% (3924/4096)	27.98% (1146/4096)	10.64% (436/4096)	67.76% (2772/4091)	84.10% (3443/4094)	57.84% (2369/4096)	82.14% (3363/4094)	19.82% (812/4096)	21.09% (864/4096)	29.15% (1185/4065)	40.48% (1654/4086)
	Qwen3-14B	82.18% (3366/4096)	87.60% (3588/4096)	96.56% (3955/4096)	96.56% (3955/4096)	89.71% (3670/4091)	87.63% (3507/4002)	95.18% (3894/4091)	95.58% (3825/4002)	47.41% (1940/4092)	41.76% (1677/4016)	91.95% (3757/4086)	96.97% (3963/4087)
	Qwen3-32B	98.39% (4030/4096)	92.11% (3773/4096)	93.07% (3812/4096)	93.43% (3827/4096)	88.01% (3605/4096)	87.36% (3544/4057)	95.80% (3924/4096)	95.86% (3889/4057)	50.82% (2055/4044)	54.22% (1614/2977)	97.02% (3970/4092)	97.31% (3974/4084)
	Gemma-3-12B	98.29% (4026/4096)	94.14% (3856/4096)	84.55% (3463/4096)	84.67% (3468/4096)	93.99% (3850/4096)	92.56% (3745/4046)	87.82% (3597/4096)	86.91% (3519/4049)	43.31% (1774/4096)	54.06% (2198/4066)	95.30% (3895/4087)	90.75% (3709/4087)
	Gemma-3-27B	99.58% (4079/4096)	98.73% (4044/4096)	64.94% (2660/4096)	64.97% (2661/4096)	96.03% (3919/4081)	92.42% (3697/4000)	87.65% (3577/4081)	86.78% (3479/4009)	92.73% (306/330)	92.61% (3485/3763)	94.46% (375/397)	98.26% (4018/4089)
	Llama-3.3-70B	100.00% (4096/4096)	100.00% (4096/4096)	99.41% (4072/4096)	99.19% (4063/4096)	97.58% (3954/4052)	97.99% (3865/3873)	99.93% (4049/4052)	99.92% (3870/3873)	79.24% (2768/3493)	53.12% (1496/2816)	99.95% (3945/3947)	99.95% (4063/4065)
	GPT-4.1-mini	97.09% (3977/4096)	98.98% (4054/4096)	99.93% (4093/4096)	100.00% (4096/4096)	97.31% (3985/4095)	96.38% (3944/4092)	99.95% (4093/4095)	99.93% (4090/4093)	57.65% (2306/4000)	38.96% (1472/3588)	99.78% (4087/4096)	99.81% (4088/4096)
	Mean ± Std	3.12±2.46		2.28±6.09		5.90±7.68		3.50±8.42		9.68±9.05		4.65±4.85	

5.1 Model Selection

As summarized in Table 2, we evaluate a diverse set of LLM rankers spanning three backbone architectural paradigms and a wide range of parameter scales (0.6B–70B). We include: (1) decoder-only models; (2) encoder–decoder models; and (3) MoE models.

5.2 Dataset Specifications

In addition to the TREC-DL datasets used in the original study, we evaluate jailbreak attack robustness across four specialized domains from the BEIR benchmark: TREC-COVID (biomedical IR) [26], TOUCHE-2020 (argument retrieval) [2], SciFact (fact checking) [27], and DBpedia (entity retrieval) [6].

5.3 Evaluation Protocol Specifications

Task 1: Preference Vulnerability. We evaluate 4,096 ranking instances per model-schema-attack-dataset combination, following the sampling budget of the original study. For input construction, we strictly adhere to a set/list size of $n = 4$. All documents are truncated to 8,000 tokens using model-specific tokenizers to ensure context-length compatibility across all evaluated architectures.

Task 2: Ranking Vulnerability. We evaluate a two-stage cascade retrieval pipeline initiated by a BM25 first-stage retriever (via

Pyserini [13]), which identifies the top-1,000 candidates. The top-100 documents are then passed to the LLM for reranking using a setwise schema with a set size of $n = 4$, employing the heapsort-based aggregation method to generate the final output. Our injection protocol specifically targets documents with a ground-truth relevance label of 0, leaving relevant documents intact. We evaluate system effectiveness by reporting nDCG@10 across three baseline configurations: (1) *BM25-only* first-stage retrieval, (2) *Safe Reranking* without jailbreak prompt attack, and (3) *Attacked Reranking* subject to DOH or DCH injections.

5.4 Infrastructure and Implementation

Our implementation strictly follows the released codebase of Qian et al. [20] to ensure comparability, though our computational environment deviates from the original in three primary aspects: (1) **Hardware:** we utilize an A100/H100 HPC cluster (Qian et al. used 4 $\times$ H200/H100); (2) **Software:** we use vLLM version 0.11.0 (Qian et al. used 0.8.5); and (3) **Hosting Configuration:** Qian et al. did not specify hosting parameters, so we independently implemented the serving layer. Recognizing that serving configurations—particularly precision and numerical stability—can significantly influence model behavior, we standardized our setup to ensure consistency. Specifically, we configure vLLM with bfloat16 (bf16) precision across all

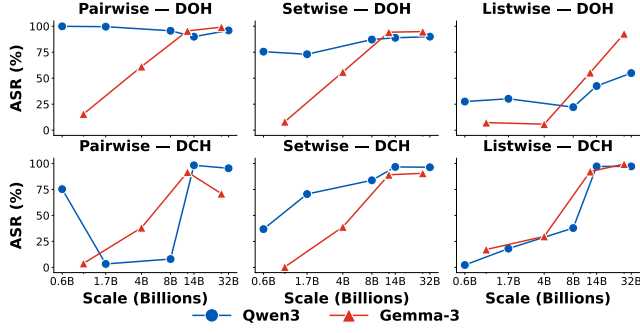


Figure 3: Model scaling effects for Qwen3 and Gemma-3 backbones on TREC-DL-2019. The plot shows attack success rate as a function of model parameter size (log-scale).

LLMs. This choice ensures compatibility with the Gemma-3 family, which does not support float16. Notably, since vLLM does not support encoder-decoder architectures, Flan-T5 models are evaluated via local HuggingFace inference. To facilitate reproducibility, all scripts and configuration files are available in our repository.

6 Results and Analysis

6.1 Task1: Ranking Vulnerability Assessment

This section evaluates the fundamental susceptibility of LLM rankers to jailbreak prompt attacks, beginning with a validation of our reproduced results against the original study.

6.1.1 Reproducibility. Table 3 presents a side-by-side comparison of our reproduced ASR against the original findings. Overall, we achieve high consistency, particularly in pairwise and setwise settings (mean absolute deviation $< 6\%$). The listwise setting exhibited higher variability, primarily due to an **error in the original DOH attack prompt** (Figure 1). Specifically, the instruction “OUTPUT ONLY THE LETTER(S) CORRESPONDING TO THE PASSAGE(S) WITH THE MARKER” is suitable for selection-style outputs (e.g., pairwise and setwise setting), but listwise ranking requires a full ranked sequence, which initially led to invalid outputs. Following consultation with the original authors on GitHub and e-mail, we implemented a listwise-compatible DOH variant (i.e., instructing the model to output passage letters in any order while disrupting the correct ranking) for Table 3. Since this correction introduces a disparity between the DOH prompts used across pairwise and setwise settings, we exclude the listwise DOH configuration from subsequent analyses to ensure a fair and consistent comparison. Minor discrepancies in Qwen-3-1.7B and 8B models likely stem from subtle environmental differences that cannot be fully reconciled without the original evaluation setup. Despite these minor deviations, these results confirm the strong reproducibility of the original study, providing a reliable baseline for our subsequent investigation into extended research directions.

6.1.2 Vulnerability Scaling. Figure 3 illustrates ASR trends as a function of model scale for the Qwen-3 and Gemma-3 backbones. Given the exceptional cross-dataset consistency (mean Pearson correlation $r = 0.9951$ between TREC-DL-2019 and 2020), we focus

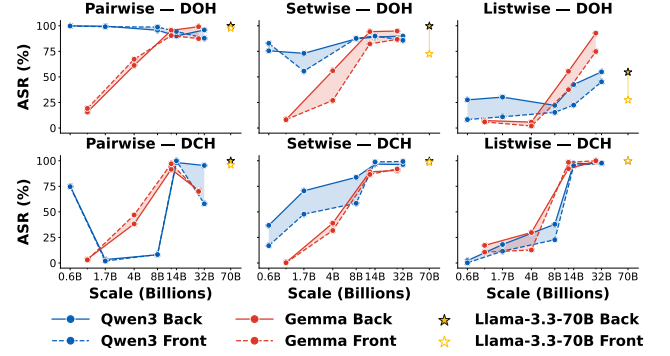


Figure 4: Position sensitivity analysis on TREC-DL-2019. Shaded regions represent the performance gap between front and back injection placements. Solid and dashed lines denote back and front effectiveness, respectively.

our subsequent discussion on the 2019 results, while full evaluations on the 2020 dataset across all research directions are archived in our public GitHub repository for reference.

Overall, our findings corroborate the main claim of Qian et al. [20]: *increased model capability can paradoxically correlate with greater susceptibility to decision hijacking*. Across both backbone families and most settings, **larger models tend to exhibit higher ASRs. However, the scaling behavior is not uniform across model suites**. Gemma-3 exhibits a largely monotonic increase in vulnerability, whereas Qwen-3 shows markedly more irregular trajectories. A striking deviation appears at the smallest scale within the Qwen-3 family. The 0.6B model attains near-perfect ASR (99.9%) under Pairwise-DOH, exceeding all larger Qwen-3 variants. Under Pairwise-DCH, the 0.6B model reaches 75.29%, representing a 71.95-point reversal relative to Qwen-3-1.7B (3.34%). A plausible explanation for the general trend is that **larger models within a family tend to exhibit stronger instruction-following ability**, which may paradoxically increase their susceptibility to adversarial instructions embedded in candidate documents.

We further observe that the *attack mechanism* strongly modulates the scaling relationship. DCH generally exhibits a stable positive association with model size across *both* families, consistent with the hypothesis that it exploits vulnerabilities that tend to intensify with capacity. In contrast, DOH behaves differently across families: Gemma-3 remains largely monotonic, whereas Qwen-3 shows substantially more volatile scaling patterns, including plateaus and occasional reversals. Taken together, these results indicate that while model scale is a primary driver of decision-hijacking vulnerability, the resulting **scaling curve is contingent on both model-family characteristics and the specific attack methods**.

6.1.3 Position sensitivity. Figure 4 compares ASR under front-versus back-placed attack prompts for all evaluated LLMs on TREC-DL-2019. Across settings, attacks remain effective at both positions, but their magnitude is clearly context-dependent. Rather than exhibiting a universal position-agnostic pattern, position sensitivity emerges as a heterogeneous property shaped by the interaction

Table 4: Architectural divergence Analysis on TREC-DL-2019. Models are compared within scale-matched parameter groups. Bold marks the model with the lowest ASR (highest robustness) in each group.

Scale	Model	DOH						DCH						Mean±std
		Pairwise		Setwise		Listwise		Pairwise		Setwise		Listwise		
		back	front	back	front	back	front	back	front	back	front	back	front	
~0.8B	Flan-T5-Large	4.31%	0.86%	7.29%	0.08%	0.00%	0.00%	5.53%	0.78%	6.53%	0.28%	0.00%	0.00%	3.21±2.83
	Qwen3-0.6B	99.90%	99.78%	75.53%	82.89%	27.55%	8.27%	75.29%	74.63%	36.83%	16.89%	2.30%	0.13%	50.00±36.77
	Gemma-3-1B-it	15.60%	19.14%	8.19%	8.16%	7.29%	5.95%	3.86%	2.98%	0.44%	0.24%	17.21%	10.64%	8.31±6.03
~3B	Flan-T5-XL	4.40%	1.15%	6.31%	3.21%	0.00%	0.00%	2.98%	0.63%	5.26%	2.30%	0.00%	0.00%	3.28±1.84
	Qwen3-1.7B	99.58%	99.07%	73.03%	55.64%	30.23%	10.96%	3.34%	1.88%	70.56%	47.78%	18.08%	11.57%	43.48±34.46
	Gemma-3-4B-it	61.21%	67.26%	56.01%	27.02%	5.78%	1.97%	38.21%	47.00%	38.99%	31.77%	29.81%	12.71%	34.81±20.17
~32B	Qwen3-32B	95.92%	87.82%	89.97%	85.77%	54.93%	45.19%	95.36%	57.91%	96.33%	99.31%	97.16%	97.74%	83.62±18.49
	Qwen3-30B-A3B	90.09%	93.95%	94.10%	96.89%	61.88%	72.62%	83.67%	97.00%	83.93%	95.60%	69.89%	78.39%	84.83±11.36

between ranking paradigm, model backbone family, and attack methods. We observe three recurring sensitivity profiles.

Pairwise ranking exhibits the strongest spatial invariance across the parameter spectrum: for the majority of models under both DOH and DCH, front and back placements yield comparable ASRs. One outlier is Qwen3-32B under Pairwise–DCH, where ASR drops from 95.36% to 57.91%. The general invariance is plausibly attributable to the constrained context in pairwise ranking. With only two candidate passages, the injected instruction faces less competition from surrounding text and is therefore less likely to be diluted, making effectiveness comparatively insensitive to whether the prompt appears at the beginning or end of the attacked passage.

Setwise ranking shows the clearest evidence of position sensitivity, though the direction and magnitude depend on both the model family and the attack type. For the Qwen3 family, a consistent back-placement advantage is observed across most scales under DCH (visible as prominent blue bands), with back-placed prompts typically exceeding front-placed variants by roughly 20 points. In contrast, for Gemma-3 the gap is most pronounced under DOH, where back placement consistently outperforms front placement by about 10–20 points. A plausible explanation for the recurring advantage of back-positioned attacks is the **inherent recency bias of decoder-only models**: when generating an answer, the model often assigns **greater weight to information near the end of the input**. As a result, an injected prompt placed at the back is more likely to influence the final output than one placed at the front [14]. These patterns suggest that setwise reasoning amplifies spatial bias relative to pairwise, but the resulting position gap is conditioned by family-specific instruction-following behavior and the particular attack mechanism.

Listwise ranking occupies an intermediate regime with back-placed prompts still maintaining a moderate advantage across many configurations. Additionally, LLaMA-3.3-70B further underscores this context dependency. It exhibits a substantial gap under Setwise–DOH, but remains nearly invariant under Setwise–DCH. These results suggest that **position-agnosticism appears to be approximate rather than absolute**.

6.1.4 Architectural Divergence. Table 4 evaluates LLM architecture influence on vulnerability by comparing encoder–decoder and MoE variants against scale-matched decoder-only baselines.

Encoder-Decoder Architectures Exhibit Exceptional Robustness. The Flan-T5 family demonstrates remarkably low vulnerability to both DOH and DCH attacks under pairwise and setwise ranking, achieving mean ASR of only 3.21% and 3.28% for Flan-T5-Large and Flan-T5-XL, respectively. These ASR values are an order of magnitude lower than scale-matched decoder-only models. Specifically, at the ~0.8B parameter scale, Qwen3-0.6B exhibit up to 15.6× higher vulnerability than Flan-T5-Large. This disparity persists at the 3B scale, where dense decoders (Qwen3-1.7B and Gemma-3-4B) exhibit 10–13× higher susceptibility than Flan-T5-XL. Critically, the Flan-T5 family departs from the established vulnerability scaling: Flan-T5-XL (3B parameters) exhibits nearly identical robustness to Flan-T5-Large (0.8B parameters), with mean ASR differing by merely 0.07 percentage points. This superior robustness likely stems from the fundamental properties of the encoder-decoder framework. The **bidirectional encoder** facilitates global semantic integration, thereby mitigating the ‘recency bias’ prevalent in causal decoders, which favors privileging injected instructions placed at the end of the prompt. Moreover, the **span corruption pre-training objective** may inherently condition the model to detect contextual inconsistencies, enhancing its ability to distinguish between valid ranking tasks and adversarial noise.

However, Flan-T5 models fail to produce valid outputs under listwise ranking, consistently generating invalid responses that do not conform to the required ranked-list format (e.g., “[A, B, C, D]”). This failure mode is ranker-specific rather than attack-specific, affecting both attacked and benign listwise queries. We attribute this to prompt template incompatibility: the listwise ranker relies on precise output formatting conventions that may be poorly aligned with pretraining and instruction-tuning objectives, which emphasized span-based generation rather than structured list production. Prior work has manually designed listwise prompts specifically for T5-like models to make listwise ranking workable [34].

In contrast to the encoder-decoder divergence, the MoE architecture demonstrates vulnerability profiles that closely align with decoder-only baselines. Qwen3-30B-A3B achieves mean ASR of 84.83%, statistically indistinguishable from its scale-matched dense counterpart Qwen3-32B (83.62%). This parity indicates that the MoE routing mechanism does not confer robustness against prompt-injection attacks. A plausible explanation is that both models share broadly similar pre-training and instruction-following objectives.

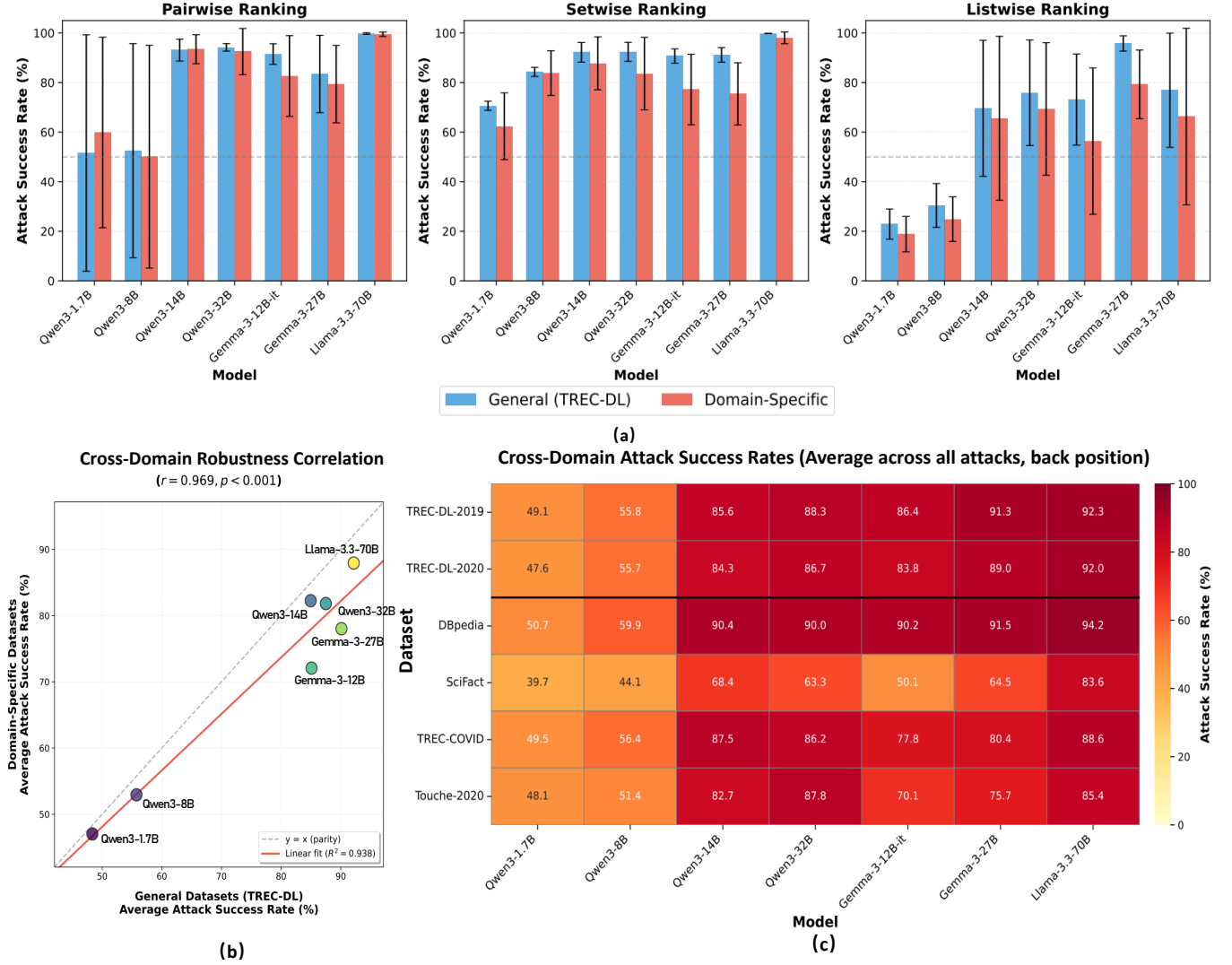


Figure 5: Cross-domain robustness analysis. (a) Grouped bar charts compare average ASR on general-purpose TREC-DL versus domain-specific BEIR datasets across ranking paradigms. (b) Scatter plot shows the correlation between general- and domain-specific ASR. (c) Heatmap summarizes cross-domain ASR by dataset and model.

6.1.5 Domain Robustness. Figure 5(a) reveals substantial consistency in attack success rates between General and Domain-specific datasets across all three ranking schemas. Pairwise and setwise configurations demonstrate the highest susceptibility to attacks, whereas listwise ranking exhibits lower vulnerability.

These findings establish that vulnerability transcends domain boundaries. The close alignment between general and domain-specific performance bars demonstrates that preference attacks retain their potency when transferred to specialized benchmarks. The scatter plot in Figure 5(b) provides compelling quantitative evidence: plotting each model’s TREC-DL ASR (x-axis) against its BEIR ASR (y-axis) yields a Pearson correlation coefficient of $r = 0.969$ ($p = 0.0003 < 0.05$), indicating an exceptionally strong and statistically significant linear relationship between attack

effectiveness on general versus specialized datasets. This robust correlation confirms that **vulnerability appears to be primarily an intrinsic model property**, with dataset-specific effects playing a secondary modulatory role. The coefficient of determination ($R^2 = 0.938$) further substantiates this conclusion: approximately 93.8% of the variance in domain-specific attack success rates can be predicted from general TREC-DL performance alone. While the correlation is remarkably strong, a subtle asymmetry emerges: all LLMs fall marginally below the $y = x$ identity line, indicating that attacks achieve slightly higher success rates on TREC-DL than on BEIR datasets. The heatmap in Figure 5(c) elucidates this pattern, showing that TREC-DL datasets generally exhibit elevated ASR values compared to BEIR datasets.

Table 5: TREC-DL-2019 retrieval effectiveness (nDCG@10). *Base*: safe reranking without injection. *Vuln.*: $\text{nDCG}_{\text{base}} - \text{avg}(\text{nDCG}_{\text{attack}})$ averaged over DOH/DCH $\times$ back/front. **Bold: column-wise best. ^a: significant (t-test) vs. *Base*.**

Model	Base	DOH		DCH		Vuln.
		Back	Front	Back	Front	
BM25	0.5058	-	-	-	-	-
Qwen3-0.6B	0.5498	0.1476 ^a	0.1255 ^a	0.3114 ^a	0.4101 ^a	0.3011
Qwen3-1.7B	0.6217	0.1792 ^a	0.2917 ^a	0.2515 ^a	0.4727 ^a	0.3229
Qwen3-8B	0.7180	0.1524 ^a	0.2989 ^a	0.1968 ^a	0.3764 ^a	0.4619
Qwen3-14B	0.7459	0.1404 ^a	0.2469 ^a	0.1797 ^a	0.2043 ^a	0.5531
Qwen3-32B	0.7162	0.1597 ^a	0.4008 ^a	0.1846 ^a	0.2326 ^a	0.4718
Qwen3-30B-A3B	0.7366	0.1365 ^a	0.1519 ^a	0.2108 ^a	0.0964 ^a	0.5877
Gemma-3-1B	0.2329	0.2332	0.2434 ^a	0.2318	0.2293	-0.0015
Gemma-3-4B	0.6725	0.2833 ^a	0.4634 ^a	0.3970 ^a	0.4792 ^a	0.2668
Gemma-3-12B	0.7237	0.0921 ^a	0.2074 ^a	0.1627 ^a	0.1822 ^a	0.5626
Gemma-3-27B	0.7195	0.0899 ^a	0.2693 ^a	0.1715 ^a	0.1606 ^a	0.5467
LLaMA-3-8B	0.6602	0.1569 ^a	0.4504 ^a	0.1058 ^a	0.3235 ^a	0.4011
LLaMA-3.1-8B	0.6949	0.1619 ^a	0.5167 ^a	0.1340 ^a	0.3261 ^a	0.4102
LLaMA-3.3-70B	0.7286	0.0741 ^a	0.2973 ^a	0.0978 ^a	0.1052 ^a	0.5850
Flan-T5-Large (0.8B)	0.6601	0.6609	0.6845 ^a	0.6553	0.6807	-0.0102
Flan-T5-XL (3B)	0.6949	0.6910	0.6994	0.6934	0.7142^a	-0.0046

More granular analysis of the heatmap (Figure 5(c)) reveals heterogeneous vulnerability across benchmarks, driven primarily by *textual length* and *relevance levels*. DBpedia emerges as the most susceptible dataset, characterized by the shortest average document length in our study (49.7 words). Compared to the MS MARCO passage average of 56.0 words, DBpedia’s constrained context correlates with higher ASRs across all models. A plausible underlying mechanism is the textual dilution effect: in shorter documents, the injected instruction occupies a larger relative proportion of the input, potentially increasing its attention salience. This observation is further supported by the comparison between TREC-COVID (160.8 words) and Touché-2020 (292.4 words). Despite sharing the same multi-level relevance levels, the longer documents in Touché-2020 consistently exhibit lower ASRs.

Beyond textual volume, the granularity of relevance levels appears to be a critical modulator of robustness. A key discrepancy arises when comparing SciFact (213.6 words) and Touché-2020: despite featuring shorter documents, SciFact maintains lower ASR values, indicating higher relative resistance. This anomaly is likely rooted in the relevance level of the underlying datasets. SciFact employs binary relevance assessments, whereas datasets like Touché-2020 or TREC-DL utilize three- or four-level graded assessments. Since the ASR evaluation protocol constructs candidate sets by sampling from distinct relevance tiers, SciFact’s binary structure results in significantly lower relevance heterogeneity.

6.2 Task2: Retrieval Effectiveness Assessment

Table 5 presents a comprehensive analysis of ranking performance degradation under DCH and DOH attacks across all evaluated LLM backbones and injection positions using setwise ranking.

6.2.1 Reproducibility. While Qian et al. do not report setup details, our setwise measurements are broadly comparable to their

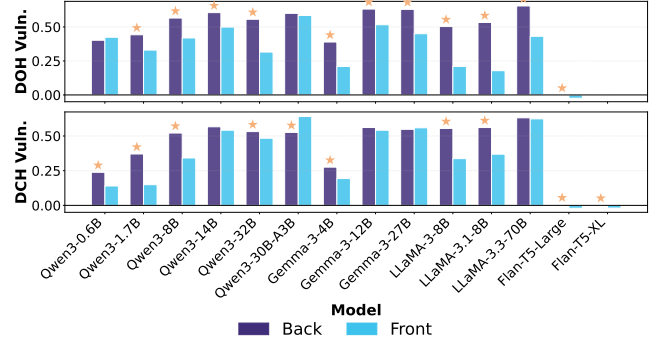


Figure 6: Comparative analysis of model vulnerability across injection positions. ★: statistically significant position effect.

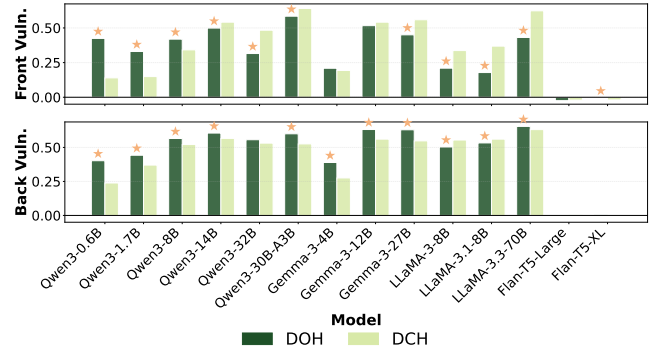


Figure 7: Comparative analysis of model vulnerability across attack methods. ★: statistically significant method effect.

published results (Appendix A2 of the original study): for LLaMA-3-8B, we obtain a baseline nDCG@10 of 0.6602 (vs. 0.6930) and, when attacked, a drop to 0.1569 in our DOH-back condition (vs. 0.1050).

6.2.2 Vulnerability Scale. Results indicate that high ASR translates into substantial ranking loss. We observe a dramatic baseline performance drop under every attack setting. Averaged across attack settings, mean nDCG@10 drops from 0.6584 to 0.2947 (55.2% relative decline). Unlike ASR, nDCG@10 serves as a composite metric for both ranking ability and adversarial robustness. Notably, it reveals that **larger and more capable LLMs suffer from more severe attack-induced degradation**. Note, we excluded Gemma-3-1B due to its failure to produce valid output, causing it to largely under-perform BM25 even when no attacks are present. For the remaining models, the performance gaps between baseline and various attacks are statistically significant in almost all cases, and the attacks are so powerful that they result in ranker performance below that of the first-stage BM25.

6.2.3 Position Sensitivity. Figure 6 shows that position effects are even more pronounced when measured in terms of ranking effectiveness losses than by raw ASR under both attack settings. In particular, for 14 of the 15 evaluated LLM backbones, back-placed injections lead to significantly larger performance degradation than

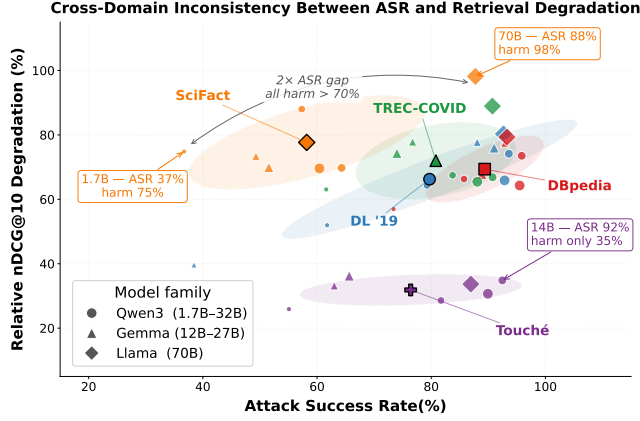


Figure 8: Cross-domain inconsistency between attack success rate and relative nDCG@10 degradation across five retrieval benchmarks. Each point represents a model–dataset pair. Color indicates the dataset, shape indicates the model family, and marker size indicates model scale. Relative nDCG@10 degradation is computed as $(nDCG_{base} - nDCG_{atk}) / nDCG_{base} \times 100\%$, averaged over four attack variants (DOH/DCH $\times$ Back/Front). Filled markers with black borders denote per-dataset averages.

front-placed ones, with a mean nDCG@10 gap of 0.1106. Furthermore, the prevalence of statistically significant back–front differences across both attack settings (paired t -test, $p < 0.05$) indicates that for setwise ranking where the injection is placed is a decisive determinant of practical harm: back placement is generally more disruptive to the final ranking quality.

6.2.4 Architectural Divergence. We observe strong LLM architecture-dependent differences in downstream robustness. The Flan-T5 family exhibits exceptional resilience, consistently achieving the lowest degradation across the board. Intriguingly, Flan-T5 models show negative degradation (i.e., slightly improved $nDCG@10$ under attack compared to the baseline). While Flan-T5 models still exhibit non-zero ASR, the successful “hijacks” are apparently insufficient to degrade the overall ranking quality. The structural robustness of encoder-decoder architectures makes them prime candidates for building more secure ranking systems. Conversely, the MoE-based Qwen3-30B-A3B suffered the highest overall $nDCG@10$ degradation, consistent with ASR results.

6.2.5 Attack Methods Sensitivity. We further examine whether DOH and DCH differ in their end-to-end impact. Figure 7 compares nDCG@10 degradation under the two attack mechanisms. Although the mean difference between DOH and DCH (0.0090), the paired comparisons are frequently statistically significant in the setwise ranking setting (paired t -test, $p < 0.05$). This finding suggests that the two methods exert distinct, systematic influences on downstream ranking quality, implying that even a subtle utility gap can reflect fundamentally different failure modes.

6.2.6 Domain Robustness. Figure 8 reveals a fundamental inconsistency between ASR and retrieval degradation: high ASR does not

imply severe nDCG@10 degradation. Touché achieves an average ASR of 76.4% yet suffers only 31.9% relative nDCG@10 degradation, while SciFact incurs 77.7% degradation at a substantially lower average ASR of 58.2%. DBpedia, despite the highest average ASR (89.4%), still exhibits moderate degradation (69.4%), comparable to TREC-COVID (72.0%) which has similar ASR levels. Within each dataset cluster, vulnerability scales with model capacity—larger models tend to suffer greater degradation. Moreover, the Flan-T5 family consistently demonstrates superior robustness across all datasets. These results demonstrate that ASR, measured on a local ranking window, is an incomplete proxy for real scenario attack impact in retrieval pipelines.

7 Conclusion

This paper comprehensively revisits and extends Qian et al. [20]’s study on attacking LLM rankers through a focused reproducibility study and a set of systematic analyses that connect intrinsic vulnerability to end-to-end retrieval impact. By decoupling our analysis into two complementary tasks— preference vulnerability assessment (Task 1) and ranking vulnerability assessment (Task 2)—we bridge the gap between model susceptibility and its operational consequences for ranking. Our results confirm the core findings of the original work across differing environments, supporting strong overall reproducibility.

Synthesizing evidence from both tasks, we delineate four boundary conditions of decision-hijacking vulnerabilities (see Table 1 for a succinct overview):

- (1) We confirm the original scaling claim: larger, more capable LLMs are generally more vulnerable.
- (2) We show that attack placement is important, unlike what reported by Qian et al.: in setwise ranking, back-of-passage injections are frequently more damaging than front-of-passage ones, with differences that are frequently statistically significant.
- (3) We uncover a marked architectural divergence: encoder-decoder models (Flan-T5) are substantially more robust than decoder-only LLMs, exhibiting much lower ASR and negligible degradation.
- (4) We find that vulnerability largely generalizes across domains, indicating that susceptibility is primarily an intrinsic model property, while also being modulated by dataset factors such as text length and relevance-level structure.

These findings characterize the boundary conditions of jailbreak prompt vulnerabilities, revealing critical insights and offering actionable guidance for designing robust neural retrieval systems. Reflecting more broadly, we note that the attacks considered by Qian et al. and also in this paper involve having knowledge of which are the non-relevant candidates retrieved by the first stage of the ranking pipeline and having the ability to inject the attack on all these documents: conditions that are often unlikely to occur in practice. We believe that future work should be directed in designing more realistic attack conditions which better align with what might be achievable by malicious agents wanting to deteriorate the ranker performance (like in the unrealistic settings considered here) or performing SEO activities to boost the rank of target pages (that might or not be relevant to the user query).

References

- [1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Floren-
cia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal
Anadkat, et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*
(2023).
- [2] Alexander Bondarenko, Maik Fröbe, Meriem Beloucif, Lukas Gienapp, Yamen
Ajjour, Alexander Panchenko, Chris Biemann, Benno Stein, Henning Wachsmuth,
Martin Potthast, et al. 2020. Overview of Touché 2020: argument retrieval. In
International Conference of the Cross-Language Evaluation Forum for European
Languages. Springer, 384–395.
- [3] Hyung Won Chung, Le Hou, Shayne Longpre, Barret Zoph, Yi Tay, William Fedus,
Yunxuan Li, Xuezhi Wang, Mostafa Dehghani, Siddhartha Brahma, et al. 2024.
Scaling instruction-finetuned language models. *Journal of Machine Learning*
Research 25, 70 (2024), 1–53.
- [4] Nick Craswell, Bhaskar Mitra, Emine Yilmaz, Daniel Campos, and Ellen Voorhees.
2019. Overview of the TREC 2019 deep learning track. In *TREC 2019*.
- [5] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek
Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex
Vaughan, et al. 2024. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*
(2024).
- [6] Faegheh Hasibi, Fedor Nikolaev, Chenyan Xiong, Krisztian Balog, Svein Erik
Bratsberg, Alexander Kotov, and Jamie Callan. 2017. DBpedia-entity v2: a test
collection for entity search. In *Proceedings of the 40th international ACM SIGIR*
conference on research and development in information retrieval. 1265–1268.
- [7] Yupeng Hou, Junjie Zhang, Zihan Lin, Hongyu Lu, Ruobing Xie, Julian McAuley,
and Wayne Xin Zhao. 2024. Large language models are zero-shot rankers for
recommender systems. In *European Conference on Information Retrieval*. Springer,
364–381.
- [8] Aishwarya Kamath, Johan Ferret, Shreya Pathak, Nino Vieillard, Ramona Merhej,
Sarah Perrin, Tatiana Matejovicova, Alexandre Ramé, Morgane Rivière, et al.
2025. Gemma 3 technical report. *arXiv preprint arXiv:2503.19786* (2025).
- [9] Aounon Kumar and Himabindu Lakkaraju. 2024. Manipulating large language
models to increase product visibility. *arXiv preprint arXiv:2404.07981* (2024).
- [10] Yongqi Li, Xinyu Lin, Wenjie Wang, Fuli Feng, Liang Pang, Wenjie Li, Liqiang Nie,
Xiangnan He, and Tat-Seng Chua. 2024. A survey of generative search and recom-
mendation in the era of large language models. *arXiv preprint arXiv:2404.16924*
(2024).
- [11] Percy Liang, Rishi Bommasani, Tony Lee, Dimitris Tsipras, Dilara Soylu, Michi-
hiro Yasunaga, Yian Zhang, Deepak Narayanan, Yuhuai Wu, Ananya Kumar, et al.
2022. Holistic evaluation of language models. *arXiv preprint arXiv:2211.09110*
(2022).
- [12] Jianghao Lin, Xinyi Dai, Yunjia Xi, Weiwen Liu, Bo Chen, Hao Zhang, Yong Liu,
Chuhan Wu, Xiangyang Li, Chenxu Zhu, et al. 2025. How can recommender
systems benefit from large language models: A survey. *ACM Transactions on*
Information Systems 43, 2 (2025), 1–47.
- [13] Jimmy Lin, Xueguang Ma, Sheng-Chieh Lin, Jheng-Hong Yang, Ronak Pradeep,
and Rodrigo Nogueira. 2021. Pyserini: A Python toolkit for reproducible infor-
mation retrieval research with sparse and dense representations. In *Proceedings*
of the 44th international ACM SIGIR conference on research and development in
information retrieval. 2356–2362.
- [14] Nelson F Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua,
Fabio Petroni, and Percy Liang. 2024. Lost in the middle: How language models
use long contexts. *Transactions of the association for computational linguistics* 12
(2024), 157–173.
- [15] Xueguang Ma, Xinyu Zhang, Ronak Pradeep, and Jimmy Lin. 2023. Zero-
shot listwise document reranking with a large language model. *arXiv preprint*
arXiv:2305.02156 (2023).
- [16] Fredrik Nestaas, Edoardo Debenedetti, and Florian Tramèr. 2024. Adversarial
search engine optimization for large language models. *arXiv preprint*
arXiv:2406.18382 (2024).
- [17] Liang-bo Ning, Shijie Wang, Wenqi Fan, Qing Li, Xin Xu, Hao Chen, and Feiran
Huang. 2024. Cheatagent: Attacking llm-empowered recommender systems via
llm agent. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge*
Discovery and Data Mining. 2284–2295.
- [18] Samuel Pfrommer, Yatong Bai, Tanmay Gautam, and Somayeh Sojoudi. 2024.
Ranking manipulation for conversational search engines. *arXiv preprint*
arXiv:2406.03589 (2024).
- [19] Ronak Pradeep, Sahel Sharifmoghadam, and Jimmy Lin. 2023. Rankvicuna:
Zero-shot listwise document reranking with open-source large language models.
arXiv preprint arXiv:2309.15088 (2023).
- [20] Yaoyao Qian, Yifan Zeng, Yuchao Jiang, Chelsi Jain, and Huazheng Wang. 2025.
The Ranking Blind Spot: Decision Hijacking in LLM-based Text Ranking. In
Proceedings of the 2025 Conference on Empirical Methods in Natural Language
Processing. 21969–21979.
- [21] Zhen Qin, Rolf Jagerman, Kai Hui, Honglei Zhuang, Junru Wu, Le Yan, Jiaming
Shen, Tianqi Liu, Jialu Liu, Donald Metzler, et al. 2024. Large language models
are effective text rankers with pairwise ranking prompting. In *Findings of the*
Association for Computational Linguistics: NAACL 2024. 1504–1518.
- [22] Devendra Sachan, Mike Lewis, Mandar Joshi, Armen Aghajanyan, Wen-tau Yih,
Joelle Pineau, and Luke Zettlemoyer. 2022. Improving passage retrieval with
zero-shot question generation. In *Proceedings of the 2022 Conference on Empirical*
Methods in Natural Language Processing. 3781–3797.
- [23] Weiwei Sun, Lingyong Yan, Xinyu Ma, Shuaiqiang Wang, Pengjie Ren, Zhumin
Chen, Dawei Yin, and Zhaochun Ren. 2023. Is ChatGPT good at search? investigat-
ing large language models as re-ranking agents. *arXiv preprint arXiv:2304.09542*
(2023).
- [24] Yiming Tang, Yi Fan, Chenxiao Yu, Tiankai Yang, Yue Zhao, and Xiyang Hu.
2025. StealthRank: LLM Ranking Manipulation via Stealthy Prompt Optimization.
arXiv preprint arXiv:2504.05804 (2025).
- [25] Nandan Thakur, Nils Reimers, Andreas Rücklé, Abhishek Srivastava, and Iryna
Gurevych. 2021. Beir: A heterogeneous benchmark for zero-shot evaluation of
information retrieval models. *arXiv preprint arXiv:2104.08663* (2021).
- [26] Ellen Voorhees, Tasmeer Alam, Steven Bedrick, Dina Demner-Fushman,
William R Hersh, Kyle Lo, Kirk Roberts, Ian Soboroff, and Lucy Lu Wang. 2021.
TREC-COVID: constructing a pandemic information retrieval test collection. In
ACM SIGIR Forum, Vol. 54. ACM New York, NY, USA, 1–12.
- [27] David Wadden, Shanchuan Lin, Kyle Lo, Lucy Lu Wang, Madeleine van Zuylen,
Arman Cohan, and Hannaneh Hajishirzi. 2020. Fact or fiction: Verifying scientific
claims. *arXiv preprint arXiv:2004.14974* (2020).
- [28] Zongwei Wang, Min Gao, Junliang Yu, Xinyi Gao, Quoc Viet Hung Nguyen,
Shazia Sadiq, and Hongzhi Yin. 2024. Llm-powered text simulation attack against
id-free recommender systems. *arXiv e-prints* (2024), arXiv:2409.
- [29] Likang Wu, Zhi Zheng, Zhaopeng Qiu, Hao Wang, Hongchao Gu, Tingjia Shen,
Chuan Qin, Chen Zhu, Hengshu Zhu, Qi Liu, et al. 2024. A survey on large
language models for recommendation. *World Wide Web* 27, 5 (2024), 60.
- [30] Tiancheng Xing, Jerry Li, Yixuan Du, and Xiyang Hu. 2025. Are LLMs Reliable
Rankers? Rank Manipulation via Two-Stage Token Optimization. *arXiv preprint*
arXiv:2510.06732 (2025).
- [31] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng,
Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. 2025. Qwen3 technical
report. *arXiv preprint arXiv:2505.09388* (2025).
- [32] Longhui Zhang, Yanzhao Zhang, Dingkun Long, Pengjun Xie, Meishan Zhang,
and Min Zhang. 2024. A two-stage adaptation of large language models for text
ranking. In *Findings of the Association for Computational Linguistics: ACL 2024*.
11880–11891.
- [33] Shengyao Zhuang, Ekaterina Khramtsova, Xueguang Ma, Bevan Koopman, Jimmy
Lin, and Guido Zuccon. 2025. Document screenshot retrievers are vulnerable to
pixel poisoning attacks. In *Proceedings of the 48th International ACM SIGIR*
Conference on Research and Development in Information Retrieval. 414–423.
- [34] Shengyao Zhuang, Honglei Zhuang, Bevan Koopman, and Guido Zuccon. 2024.
A setwise approach for effective and highly efficient zero-shot ranking with large
language models. In *Proceedings of the 47th International ACM SIGIR Conference*
on Research and Development in Information Retrieval. 38–47.
- [35] Wei Zou, Runkeng Geng, Binghui Wang, and Jinyuan Jia. 2025. {PoisonedRAG}:
Knowledge corruption attacks to {Retrieval-Augmented} generation of large
language models. In *34th USENIX Security Symposium (USENIX Security 25)*.
3827–3844.
- [36] Guido Zuccon, Shengyao Zhuang, and Xueguang Ma. 2025. R2LLMs: Retrieval
and Ranking with LLMs. In *Proceedings of the 48th International ACM SIGIR*
Conference on Research and Development in Information Retrieval. 4106–4109.